

# zkPoP v1

A machine-checked zero-knowledge *proof-of-power* layer for the Lean 4 formalization of  
arXiv:2608.13759

Aristotle (Harmonic)

26 August 2026

## Abstract

This report documents the **zkPoP** layer of the Lean 4 repository that formalizes arXiv:2608.13759 (*GPU Offload in Rust: Portable, Safe, and Fast*) for the target device recorded in the repository, an NVIDIA GeForce RTX 3080 Ti (12288 MiB of global memory). The layer, living in `RequestProject/ZkPoP/`, is 1853 lines of Lean across nine modules and 83 theorem-level declarations, and it develops, from first principles and without any additional axioms: a typed object language with a verified type checker; a faithful arithmetization of that checker; a compiler from polynomial constraint systems to flat gate circuits with a satisfiability-preservation theorem in both directions; a catalogue of five verified gadgets; Pedersen commitments with perfect hiding and a binding reduction to the discrete logarithm; the Schnorr  $\Sigma$ -protocol with completeness, special soundness (witness extraction) and honest-verifier zero knowledge; a *self-hosting* universal circuit that verifies any gate circuit given as a private witness, together with two-level recursion; and Reed–Solomon/Shamir sharding with proofs of durability and privacy. Everything is closed by one end-to-end theorem, `ZkPoP.end_to_end_recursive`. The library builds with `lake build`, contains no `sorry`, and every headline theorem’s axiom closure is contained in Lean’s three standard axioms.

## Contents

|          |                                                          |          |
|----------|----------------------------------------------------------|----------|
| <b>1</b> | <b>Scope and reading guide</b>                           | <b>2</b> |
| 1.1      | What this document is . . . . .                          | 2        |
| 1.2      | Provenance . . . . .                                     | 2        |
| 1.3      | The name . . . . .                                       | 2        |
| 1.4      | Module map . . . . .                                     | 3        |
| <b>2</b> | <b>Phase 0: the semantic bridge</b>                      | <b>3</b> |
| 2.1      | Types, terms, values . . . . .                           | 3        |
| 2.2      | The bridge theorem . . . . .                             | 3        |
| 2.3      | The encoding is injective . . . . .                      | 3        |
| 2.4      | Index-normal form . . . . .                              | 4        |
| <b>3</b> | <b>Phase I: arithmetization of the checker</b>           | <b>4</b> |
| <b>4</b> | <b>Phase II: constraint systems, flattening, gadgets</b> | <b>5</b> |
| 4.1      | Two levels of representation . . . . .                   | 5        |
| 4.2      | Tseitin-style flattening . . . . .                       | 5        |
| 4.3      | The gadget catalogue . . . . .                           | 5        |
| <b>5</b> | <b>Phase III: commitment and extraction</b>              | <b>6</b> |
| 5.1      | Pedersen commitments . . . . .                           | 6        |
| 5.2      | The Schnorr $\Sigma$ -protocol . . . . .                 | 7        |

|          |                                                           |           |
|----------|-----------------------------------------------------------|-----------|
| <b>6</b> | <b>Phase IV: self-hosting and universal verification</b>  | <b>7</b>  |
| 6.1      | A trace-table checker . . . . .                           | 7         |
| 6.2      | The check is not vacuous . . . . .                        | 8         |
| 6.3      | Two-level recursion . . . . .                             | 8         |
| 6.4      | Aggregation . . . . .                                     | 8         |
| <b>7</b> | <b>Phase V: deployment, sharding and audit</b>            | <b>8</b>  |
| 7.1      | The artifact hides the witness . . . . .                  | 8         |
| 7.2      | The end-to-end theorem . . . . .                          | 9         |
| 7.3      | Durability and privacy of the deployed artifact . . . . . | 9         |
| 7.4      | Axiom audit . . . . .                                     | 9         |
| <b>8</b> | <b>What is claimed, and what is not</b>                   | <b>10</b> |
| <b>9</b> | <b>Reproduction</b>                                       | <b>11</b> |

# 1 Scope and reading guide

## 1.1 What this document is

This is a v1 write-up of a body of Lean 4 code that already exists in this repository. Every statement quoted below is a declaration in `RequestProject/ZkPoP/`, reproduced from the source; the file and name are given so that each claim can be checked directly against the machine-checked artifact. Nothing in this document is asserted informally that is not proved in the code, and Section 8 lists precisely what is *not* claimed.

## 1.2 Provenance

|                                        |                                                                  |
|----------------------------------------|------------------------------------------------------------------|
| Repository                             | Lean 4 formalization of arXiv:2608.13759                         |
| Layer                                  | <code>RequestProject/ZkPoP/</code>                               |
| Umbrella module                        | <code>RequestProject/ZkPoP.lean</code>                           |
| Lean toolchain                         | <code>leanprover/lean4:v4.28.0</code>                            |
| Mathlib                                | <code>8f9d9cff6bd728b17a24e163c9402775d9e6a365</code>            |
| Target device (recorded in the export) | NVIDIA GeForce RTX 3080 Ti, 12288 MiB global                     |
| Build                                  | <code>lake build RequestProject.ZkPoP → exit 0, 3112 jobs</code> |
| Occurrences of <code>sorry</code>      | 0                                                                |
| Additional axioms                      | none                                                             |

The device line is a recorded target, not a measurement: the zkPoP layer is pure Lean and does not execute on the GPU. The connection to the device is that the repository’s verified kernel-offload pipeline — the actual subject of arXiv:2608.13759 — targets that card, and zkPoP is the layer that lets an artifact of that pipeline be published together with a proof of its correctness that discloses nothing else.

## 1.3 The name

A *proof of power* in the sense used here is a proof that the prover possesses a witness of computational work — a satisfying assignment of a circuit, an opening of a commitment, a well-typed program with a certified evaluation — and that discloses nothing about that witness beyond its existence. The three ingredients are the arithmetization of the check (Sections 2–4), the commitment and extraction machinery (Section 5), and the recursion that lets one verifier check another (Section 6).

## 1.4 Module map

| module              | lines | defs | thms | role                                        |
|---------------------|-------|------|------|---------------------------------------------|
| ZkPoP/Lang.lean     | 274   | 8    | 13   | Phase 0: object language, checker, encoding |
| ZkPoP/TypeSat.lean  | 172   | 1    | 7    | Phase I: arithmetization of the checker     |
| ZkPoP/Circuit.lean  | 430   | 15   | 18   | Phase II: polynomials, gates, flattening    |
| ZkPoP/Catalog.lean  | 381   | 23   | 18   | Phase II: the verified gadget catalogue     |
| ZkPoP/Pedersen.lean | 79    | 1    | 4    | Phase III: commitments                      |
| ZkPoP/Schnorr.lean  | 105   | 7    | 4    | Phase III: the $\Sigma$ -protocol           |
| ZkPoP/SelfHost.lean | 230   | 15   | 10   | Phase IV: universal circuit, recursion      |
| ZkPoP/Shards.lean   | 68    | 2    | 3    | Phase V: durability and privacy of shards   |
| ZkPoP/Audit.lean    | 114   | 1    | 6    | Phase V: headline theorems and axiom audit  |
| total               | 1853  | 73   | 83   |                                             |

Counts are of top-level `def`/`structure`/`inductive` and `theorem`/`lemma`/`example` declarations respectively; the umbrella file `RequestProject/ZkPoP.lean` (27 lines) only re-exports.

## 2 Phase 0: the semantic bridge

Everything downstream reasons about an *arithmetized* type check. That is only meaningful if the checker being arithmetized is itself sound with respect to an independent evaluator. Phase 0 fixes both and proves the bridge between them.

### 2.1 Types, terms, values

```
inductive Ty where
| nat : Ty
| bool : Ty
| vec :  $\mathbb{N} \rightarrow \text{Ty} \rightarrow \text{Ty}$ 
```

Terms `Tm` are literals, `tru`, `fls`, `add`, `mul`, `eqz` (test for zero), `ite` and `rep` (replication into a vector); values `Val` carry naturals, booleans and vectors, and `Val.HasTy` relates the two. The checker `Tm.infer : Tm  $\rightarrow$  Option Ty` and the evaluator `Tm.eval : Tm  $\rightarrow$  Option Val` are defined independently by structural recursion.

### 2.2 The bridge theorem

**Theorem 2.1** (`ZkPoP.Tm.checker_sound`, `ZkPoP/Lang.lean`). *Acceptance by the checker guarantees that the evaluator returns a value, and that the value has the inferred type.*

```
theorem checker_sound :
 $\forall (t : \text{Tm}) (T : \text{Ty}), t.\text{infer} = \text{some } T \rightarrow \exists v, t.\text{eval} = \text{some } v \wedge \text{Val.HasTy } v T$ 
```

This is the non-negotiable interface of the framework: no later phase is permitted to talk about `eval` except through this theorem. Its axiom closure is `[propext]` alone.

### 2.3 The encoding is injective

The arithmetization represents a type by a field element (its *tag*) or by a flat wire vector. Both representations must determine the type, or the circuit could confuse two types.

```
def Ty.tag : Ty  $\rightarrow$   $\mathbb{N}$ 
| .nat => 0
| .bool => 1
| .vec n A => 2 + Nat.pair n A.tag
```

```

def Ty.wires : Ty → List ℕ
| .nat => [0]
| .bool => [1]
| .vec n A => 2 :: n :: A.wires

theorem Ty.tag_injective : Function.Injective tag
theorem Ty.wires_injective_flat : Function.Injective wires

```

Tag uniqueness is proved for the whole, infinite type universe — the tag alphabet is realised in  $\mathbb{N}$ , the canonical representatives of the field elements, rather than in a bounded fragment. Flat-wire injectivity is derived from a stronger, prefix-freeness statement (`Ty.wires_append_inj`): reading a type off a wire stream determines both the type and the remainder of the stream, so there are no junk wire vectors.

## 2.4 Index-normal form

Type indices are Lean naturals, so semantically equal indices are *definitionally* equal and receive the same encoding. In the Lean development this is not an argument but a `rfl`:

```

theorem Ty.vec_index_normal (A : Ty) : Ty.vec (2 + 3) A = Ty.vec 5 A := rfl
theorem Ty.tag_index_normal (A : Ty) : tag (.vec (2 + 3) A) = tag (.vec 5 A) := rfl
theorem Ty.wires_index_normal (A : Ty) : wires (.vec (2 + 3) A) = wires (.vec 5 A) := rfl

```

The circuit is therefore insensitive to conversion, exactly as the Lean checker is.

## 3 Phase I: arithmetization of the checker

`ZkPoP.typeSat` :  $\text{Tm} \rightarrow \mathbb{N} \rightarrow \text{Prop}$  is the constraint system: `typeSat t τ` says that  $\tau$  is a tag that the arithmetized checker accepts for  $t$ . Its clauses are the gate constraints; each is exposed as an `Iff.rfl` lemma, so the arithmetic content is visible in the statement rather than hidden in a definition.

```

theorem typeSat_add (a b : Tm) τ( : ℕ) :
  typeSat (.add a b) τ ↔ ∃ ta tb, typeSat a ta ∧ typeSat b tb ∧
    ta = 0 ∧ tb = 0 ∧ τ = 0 := Iff.rfl

theorem typeSat_ite (c a b : Tm) τ( : ℕ) :
  typeSat (.ite c a b) τ ↔ ∃ tc ta tb, typeSat c tc ∧ typeSat a ta ∧ typeSat b tb ∧
    tc = 1 ∧ ta = tb ∧ τ = ta := Iff.rfl

```

So `.add/.mul` force both operand tags to `Ty.nat = 0`, and `.ite` forces the condition tag to `Ty.bool = 1` and the two branch tags to agree.

**Theorem 3.1** (Faithfulness, `ZkPoP.typeSat_iff`). *The constraint system accepts exactly what the Lean checker accepts.*

```

theorem typeSat_iff (t : Tm) τ( : ℕ) :
  typeSat t τ ↔ ∃ T, t.infer = some T ∧ τ = T.tag

```

Two consequences close Phase I. Uniqueness of the accepted tag, `typeSat_unique` : `typeSat t τ → typeSat t τ' → τ = τ'`, says a satisfying assignment cannot claim two types; and, combining faithfulness with tag injectivity and the Phase 0 bridge,

**Theorem 3.2** (`ZkPoP.typeSat_eval_sound`). *A satisfied arithmetized type check certifies the evaluator's result.*

```

theorem typeSat_eval_sound {t : Tm} {T : Ty} (h : typeSat t T.tag) :
  ∃ v, t.eval = some v ∧ Val.HasTy v T

```

This is the first half of the end-to-end theorem of Section 7.

## 4 Phase II: constraint systems, flattening, gadgets

### 4.1 Two levels of representation

High-level systems are lists of polynomial expressions required to vanish; low-level circuits are lists of three-address gates.

```

inductive Poly (F : Type*) where
  | var (i : ℕ) | const (c : F) | add (p q : Poly F) | mul (p q : Poly F)

structure PolySystem (F : Type*) where eqs : List (Poly F)
def PolySystem.Sat (S : PolySystem F) σ( : ℕ → F) : Prop := ∀ p ∈ S.eqs, p.eval σ = 0

inductive Gate (F : Type*) where
  | add (o a b : ℕ) | mul (o a b : ℕ) | const (o : ℕ) (c : F) | eqZero (a : ℕ)

structure GateCircuit (F : Type*) where gates : List (Gate F)
def GateCircuit.Sat (C : GateCircuit F) σ( : ℕ → F) : Prop := ∀ g ∈ C.gates, g.Holds σ

```

`Poly.bound p n` says every variable of `p` is a wire below `n`; it is the hypothesis that makes scratch-wire allocation safe.

### 4.2 Tseitin-style flattening

`Poly.flat` compiles a polynomial into gates, allocating fresh wires from `n` upwards and returning the output wire, the gates and the next free index.

```

def Poly.flat : Poly F → ℕ → FlatOut F
  | .var i, n => (i, [], )n
  | .const c, n => (n, [.const n c], n + 1)
  | .add p q, n =>
    let a := p.flat n; let b := q.flat a.next
    (b.next, a.gates ++ b.gates ++ [.add b.next a.out b.out], b.next + 1)
  | .mul p q, n =>
    let a := p.flat n; let b := q.flat a.next
    (b.next, a.gates ++ b.gates ++ [.mul b.next a.out b.out], b.next + 1)

```

The wire discipline is proved, not assumed: `flat_next_ge` ( $n \leq \text{next}$ ), `flat_out_lt` (the output wire is allocated) and `flat_gates_lt` (every gate touches wires in the allocated window) together justify that separate subtrees do not clobber each other's scratch space. On top of them:

```

theorem Poly.flat_sound : ∀ (p : Poly F) (n : ℕ) σ( : ℕ → F),
  ∀( g ∈ (p.flat n).gates, g.Holds σ) → σ (p.flat n).out = p.eval σ

theorem Poly.flat_complete : ∀ (p : Poly F) (n : ℕ) σ( : ℕ → F), p.bound n →
  ∃ σ', ∀( i < n, σ' i = σ i) ∧ ∀( i, (p.flat n).next ≤ i → σ' i = σ i) ∧
  ∀( g ∈ (p.flat n).gates, g.Holds σ') ∧ σ' (p.flat n).out = p.eval σ

```

Completeness is stated in the strong form that matters for composition: the extending assignment agrees with  $\sigma$  both *below* the allocation window and *above* it. The two directions combine into the compiler-correctness statement:

**Theorem 4.1** (Compilation preserves satisfiability, `ZkPoP.flatSys_sat_iff`).

```

theorem flatSys_sat_iff (S : PolySystem F) (n : ℕ) (hb : ∀ p ∈ S.eqs, p.bound n)
  σ( : ℕ → F) :
  S.Sat σ ↔ ∃ σ', ∀( i < n, σ' i = σ i) ∧ (S.flatSys n).Sat σ'

```

### 4.3 The gadget catalogue

Five gadgets are built and verified, spanning the three kinds a practical catalogue needs (a function, a relation, a data structure). `ZkPoP.Circ.catalog` collects them, and `Circ.catalog_names` / `Circ.catalog_kinds` pin the catalogue's contents by rfl.

| gadget        | kind           | guarantee                                                                                                                                         |
|---------------|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| isZeroSys     | function       | isZeroSys_sound: $\sigma_1 = 1$ if $\sigma_0 = 0$ , else 0, using the standard inverse-hint wire $\sigma_2$ ; isZeroSys_complete builds the hint. |
| muxSys        | function       | muxSys_sound: the selector is a bit and $\sigma_3 = \sigma_1$ or $\sigma_2$ accordingly; muxSys_complete conversely.                              |
| hornerSys     | function       | hornerSys_sound: the trace wires compute the Horner value of the coefficient list; hornerEqs_complete produces the trace.                         |
| rootSys       | relation       | rootSys_sound: satisfaction forces $\sigma_0$ to be a root of the polynomial; rootSys_complete conversely.                                        |
| merklePathSys | data structure | merklePath_sound: the constraints compute the fold of the path; merklePath_bits: every direction wire is forced to be a bit.                      |

For example the zero test, whose two equations are  $x \cdot h + b - 1 = 0$  and  $x \cdot b = 0$ :

```
theorem isZeroSys_sound  $\sigma$  ( :  $\mathbb{N} \rightarrow F$ ) (h : (isZeroSys : PolySystem F).Sat  $\sigma$ ) :  

 $\sigma$  1 = if  $\sigma$  0 = 0 then 1 else 0
```

Finally the two layers are joined: isZeroSys\_flat\_sat\_iff and muxSys\_flat\_sat\_iff instantiate flatSys\_sat\_iff at catalogued gadgets, so a catalogue entry may be used either as a polynomial system or as the flat circuit it compiles to.

*Remark 4.2.* hashF a b = a\*a + (b\*b + a\*b) in the Merkle gadget is an explicit algebraic compression function chosen to be cheap in-circuit. The Merkle theorems are statements about that function; they are not, and do not pretend to be, collision-resistance claims.

## 5 Phase III: commitment and extraction

Throughout,  $q$  is a prime,  $G$  is a module over  $\mathbb{Z}/q$ , and  $g, h \in G$  are the two bases. Nothing about  $G$  beyond the module structure is assumed.

### 5.1 Pedersen commitments

```
def pedersen (g h : G) (m r : ZMod q) : G := m • g + r • h
```

**Theorem 5.1** (Perfect hiding, ZkPoP.pedersen\_hiding).

```
theorem pedersen_hiding (g h : G) (hsurj : Function.Surjective (fun r : ZMod q => r • h))  

(m m' r : ZMod q) :  $\exists r', \text{pedersen } g \ h \ m \ r = \text{pedersen } g \ h \ m' \ r'$ 
```

Whenever the randomness acts surjectively, the commitment of *any* message can be explained by *some* randomness, so the commitment carries no information about  $m$ . The distributional form, pedersen\_hiding\_bijective, strengthens this: for a fixed message the commitment is a bijection of the randomness, hence uniformly distributed and independent of the message.

**Theorem 5.2** (Binding as a reduction, ZkPoP.pedersen\_binding).

```
theorem pedersen_binding {g h : G} (hg : g  $\neq$  0) {m r m' r' : ZMod q}  

(hsq : pedersen g h m r = pedersen g h m' r') (hm : m  $\neq$  m') :  

 $\exists x : \text{ZMod } q, h = x \cdot g$ 
```

Two openings of one commitment with different messages *exhibit* the discrete logarithm of  $h$  in base  $g$ . This is a reduction, not an unconditional impossibility claim: breaking binding breaks discrete log in  $G$ .

## 5.2 The Schnorr $\Sigma$ -protocol

The prover holds an opening  $(m, r)$  of  $C = m \cdot g + r \cdot h$  and proves knowledge of it: announcement  $t = a \cdot g + b \cdot h$ , challenge  $c$ , responses  $z = a + cm$ ,  $w = b + cr$ , verification  $z \cdot g + w \cdot h = t + c \cdot C$ .

```
def verify (g h C : G) (T : Transcript G q) : Prop := T.z • g + T.w • h = T.t + T.c • C

theorem completeness (g h : G) (m r a b c : ZMod q) :
  verify g h (pedersen g h m r) (prove g h m r a b c)
```

**Theorem 5.3** (Special soundness / extraction, `ZkPoP.Schnorr.special_soundness`).

```
theorem special_soundness (g h C : G) (t : G) (1c 2c 1z 2z 1w 2w : ZMod q) (hne : 1c ≠ 2c)
  (1h : verify g h C (t, 1c, 1z, 1w)) (2h : verify g h C (t, 2c, 2z, 2w)) :
  pedersen g h (extract 1c 2c 1z 2z 1w 2w).1 (extract 1c 2c 1z 2z 1w 2w).2 = C
```

The extractor is explicit — it divides the response differences by the challenge difference, which is invertible because  $\mathbb{Z}/q$  is a field — and the theorem says its output really opens  $C$ . This extractability is the technical core of the “proof of power”: an accepting prover that can answer two challenges provably possesses the witness.

**Theorem 5.4** (Honest-verifier zero knowledge).

```
theorem simulate_verifies (g h C : G) (c z w : ZMod q) :
  verify g h C (simulate g h C c z w)
theorem accepting_eq_simulated (g h C : G) (T : Transcript G q) (hT : verify g h C T) :
  T = simulate g h C T.c T.z T.w
```

(`ZkPoP.Schnorr.simulate_verifies`, `ZkPoP.Schnorr.accepting_eq_simulated`.) Together these say the accepting transcripts are *exactly* the simulated ones. The simulator does not use the witness, so a transcript is not evidence about it: the two distributions coincide as sets, pointwise.

## 6 Phase IV: self-hosting and universal verification

### 6.1 A trace-table checker

The universal circuit does not mention the circuit it verifies; that circuit is the witness. Each gate is loaded into a row of four selectors and four values, and the row is subject to a fixed set of algebraic constraints.

```
structure Row (F : Type*) where sAdd sMul sConst sEq vo va vb k : F

def rowOK (R : Row F) : Prop :=
  R.sAdd * (R.vo - R.va - R.vb) = 0 ∧ R.sMul * (R.vo - R.va * R.vb) = 0 ∧
  R.sConst * (R.vo - R.k) = 0 ∧ R.sEq * R.va = 0

def load σ ( : ℕ → F) : Gate F → Row F
| .add o a b => ⟨1, 0, 0, 0, σ o, σ a, σ b, 0⟩
| .mul o a b => ⟨0, 1, 0, 0, σ o, σ a, σ b, 0⟩
| .const o c => ⟨0, 0, 1, 0, σ o, 0, 0, c⟩
| .eqZero a => ⟨0, 0, 0, 1, 0, σ a, 0, 0⟩

def univCircuit (rows : List (Row F)) : Prop := ∀ R ∈ rows, rowOK R
```

**Theorem 6.1** (Self-hosting, `ZkPoP.univ_iff`).

```
theorem univ_iff (C : GateCircuit F) σ ( : ℕ → F) : univCircuit (univRows C σ) ↔ C.Sat σ
```

Both directions matter: soundness (the universal check is not weaker than the circuit) and completeness (it is not stronger, so honest provers are never rejected). The row-level lemma `rowOK_load_iff` carries the content.

## 6.2 The check is not vacuous

A universal verifier that accepted everything would satisfy half of `univ_iff` trivially, so the development pins a concrete discriminating instance:

```
theorem demo_discriminates : demoCircuit.Sat demoGood  $\wedge$   $\neg$  demoCircuit.Sat demoBad
```

with `demo_accepted` and `demo_rejected` proved directly about `univCircuit`. Likewise Phase I is pinned by a concrete term `demoTerm = if isZero 0 then 1 else 2 + 3`, for which `infer` returns `Ty.nat`, `eval` returns `Val.nat 1`, and the arithmetized check is satisfied at `Ty.tag nat` — all by `rfl`.

## 6.3 Two-level recursion

The universal constraints are themselves a `PolySystem (univSys)`, so they can be compiled by the Phase II flattener into a gate circuit — a verifier for the verifier.

**Theorem 6.2** (`ZkPoP.univ_two_level`, `ZkPoP.univ_two_level_circuit`).

```
theorem univ_two_level (u :  $\mathbb{N} \rightarrow F$ ) (g : Gate F) :
  g.Holds u  $\leftrightarrow$ 
     $\exists$   $\sigma'$ ,  $\forall$  (i < 8,  $\sigma'$  i = rowAssign (load u g) i)  $\wedge$ 
      ((univSys : PolySystem F).flatSys 8).Sat  $\sigma'$ 

theorem univ_two_level_circuit (C : GateCircuit F) (u :  $\mathbb{N} \rightarrow F$ ) :
  C.Sat u  $\leftrightarrow$   $\forall$  g  $\in$  C.gates,
     $\exists$   $\sigma'$ ,  $\forall$  (i < 8,  $\sigma'$  i = rowAssign (load u g) i)  $\wedge$ 
      ((univSys : PolySystem F).flatSys 8).Sat  $\sigma'$ 
```

The inner level attests that the circuit under verification is satisfied by the private assignment; the outer level is a flat gate circuit, obtained mechanically from the universal constraint system, attesting that the row check itself was performed correctly. The equivalence is an  $\leftrightarrow$ , so recursion loses nothing in either direction.

## 6.4 Aggregation

```
theorem library_checked (lib : List (GateCircuit F  $\times$   $\mathbb{N}(\rightarrow F)$ )) :
  univCircuit (libraryRows lib)  $\leftrightarrow$   $\forall$  p  $\in$  lib, p.1.Sat p.2
```

One universal check certifies an entire library of circuits with their assignments — the batching statement a deployment needs in order to publish many artifacts under a single proof.

# 7 Phase V: deployment, sharding and audit

## 7.1 The artifact hides the witness

**Theorem 7.1** (`ZkPoP.artifact_hides_witness`).

```
theorem artifact_hides_witness (g h : G)
  (hsurj : Function.Surjective (fun r : ZMod q => r • h))
  (m r m' : ZMod q) (T : Schnorr.Transcript G q)
  (hT : Schnorr.verify g h (pedersen g h m r) T) :
   $\exists$  r' : ZMod q, pedersen g h m' r' = pedersen g h m r  $\wedge$ 
    Schnorr.verify g h (pedersen g h m' r') T
```

The published artifact is the commitment together with an accepting transcript. The theorem says that for *every* alternative witness  $m'$  there is randomness  $r'$  producing the same commitment and leaving the same transcript accepting. The artifact is therefore exactly as consistent with any other witness as with the real one, and so discloses nothing about which witness the prover holds.



## 7.2 The end-to-end theorem

**Theorem 7.2** (ZkPoP.end\_to\_end\_recursive).

```

theorem end_to_end_recursive
  {t : Tm} {T : Ty} (hType : typeSat t T.tag)
  {F : Type*} [CommRing F] (C : GateCircuit F) (u : ℕ → F)
  (houter : ∀ gate ∈ C.gates, ∃ σ', ∀( i < 8, σ' i = rowAssign (load u gate) i) ∧
    ((univSys : PolySystem F).flatSys 8).Sat σ')
  (g h Cm tAnn : G) (1C 2C 1Z 2Z 1W 2W : ZMod q) (hne : 1C ≠ 2C)
  (1h : Schnorr.verify g h Cm (tAnn, 1C, 1Z, 1)w)
  (2h : Schnorr.verify g h Cm (tAnn, 2C, 2Z, 2)w) :
  ∃( v, t.eval = some v ∧ Val.HasTy v T) ∧ C.Sat u ∧
    pedersen g h (Schnorr.extract 1C 2C 1Z 2Z 1W 2W).1
    (Schnorr.extract 1C 2C 1Z 2Z 1W 2W).2 = Cm

```

Read as a deployment rule: from (i) a satisfying assignment of the arithmetized type check, (ii) satisfaction of the outer, flattened universal circuit on the loaded rows of every gate of the inner circuit, and (iii) two accepting transcripts sharing an announcement and differing in challenge, the verifier is entitled to conclude that the evaluator produces a value of the claimed type, that the inner circuit really is satisfied by the private assignment, and that the prover knows an opening of the commitment. All three hypotheses are checkable by the verifier; none of them exposes  $u$  or  $(m, r)$ .

## 7.3 Durability and privacy of the deployed artifact

The artifact is encoded as a polynomial  $p$  over a field  $F$  and distributed: carrier  $i$  receives  $p(v_i)$ .

**Theorem 7.3** (Durability, ZkPoP.shards\_recover).

```

theorem shards_recover (p : F[X]) (v : ι → F) (s : Finset ι) (hv : Set.InjOn v s)
  (hdeg : p.degree < s.card) : shardsDecode s v (shard p v) = p

```

Any set of carriers whose cardinality exceeds  $\deg p$  reconstructs the artifact exactly (Reed–Solomon decoding, via Lagrange interpolation).

**Theorem 7.4** (Privacy, ZkPoP.shards\_hide\_payload).

```

theorem shards_hide_payload (v : ι → F) (s : Finset ι) (o : ι) (ho : o ∉ s)
  (hv : Set.InjOn v (insert o s)) (shares : ι → F) (y : F) :
  ∃ q : F[X], q.degree < (s.card + 1 : ℕ) ∧ ∀( i ∈ s, q.eval (v i) = shares i) ∧
    q.eval (v o) = y

```

If the secret index  $o$  is excluded from the shard set  $s$ , then for *every* candidate value  $y$  there is an artifact of the same size producing exactly the observed shards on  $s$  and having  $y$  as its secret: sub-threshold shards reveal nothing (Shamir). `shards_indistinguishable` restates this in two-value form.

## 7.4 Axiom audit

ZkPoP/Audit.lean ends with `#print axioms` for every headline theorem. The following table is the output of `lake build RequestProject.ZkPoP.Audit` in this repository.

| theorem                                                        | axiom closure                                                                  |
|----------------------------------------------------------------|--------------------------------------------------------------------------------|
| <code>Tm.checker_sound</code>                                  | <code>propext</code>                                                           |
| <code>Ty.wires_injective_flat</code>                           | <code>propext</code>                                                           |
| <code>univ_iff</code>                                          | <code>propext</code> , <code>Quot.sound</code>                                 |
| <code>library_checked</code>                                   | <code>propext</code> , <code>Quot.sound</code>                                 |
| <code>demo_accepted</code> , <code>demo_rejected</code>        | <code>propext</code> , <code>Quot.sound</code>                                 |
| <code>Ty.tag_injective</code> , <code>typeSat_iff</code>       | <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code> |
| <code>flatSys_sat_iff</code>                                   | <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code> |
| <code>isZeroSys_sound</code> , <code>muxSys_sound</code>       | <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code> |
| <code>hornerSys_sound</code> , <code>rootSys_sound</code>      | <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code> |
| <code>merklePath_sound</code>                                  | <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code> |
| <code>pedersen_hiding</code> , <code>pedersen_binding</code>   | <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code> |
| <code>Schnorr.completeness</code>                              | <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code> |
| <code>Schnorr.special_soundness</code>                         | <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code> |
| <code>univ_two_level</code>                                    | <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code> |
| <code>shards_recover</code> , <code>shards_hide_payload</code> | <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code> |
| <code>artifact_hides_witness</code>                            | <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code> |
| <code>end_to_end_recursive</code>                              | <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code> |

No axiom outside Lean’s standard three occurs anywhere in the closure, and several theorems use strictly fewer. No axiom declaration and no `@[implemented_by]` attribute is introduced by the layer.

## 8 What is claimed, and what is not

The value of a formalization lies as much in its honest boundary as in its theorems.

1. **No cryptographic hardness is proved.** Binding is a *reduction*: it exhibits the discrete logarithm from a double opening. Whether discrete log is hard in a given  $G$  is an assumption, outside the formalization.
2. **hashF is not a cryptographic hash.** It is an explicit low-degree algebraic compression function; the Merkle theorems are about it as written.
3. **Zero knowledge is honest-verifier and pointwise.** `accepting_eq_simulated` identifies accepting transcripts with simulated ones as objects; no probability space is constructed and no statistical-distance statement is made. Likewise `artifact_hides_witness` is a per-witness consistency statement, which is the perfect-hiding form appropriate here.
4. **No Fiat–Shamir transform is formalized.** The protocol is interactive; the hash-based non-interactive variant, and its random-oracle analysis, are not in scope.
5. **The object language is small.** It has naturals, booleans and length-indexed vectors — enough to make the semantic bridge, the tag encoding and index-normal form meaningful, and not intended as a model of a full programming language.
6. **No GPU execution.** The layer is pure Lean. The NVIDIA GeForce RTX 3080 Ti appears here as the target device recorded by the repository’s export pipeline, which is what the zkPoP layer would attest about; the zkPoP theorems themselves are device-independent.
7. **Soundness rests on Lean’s kernel and Mathlib.** The audit above bounds the trusted base to Lean’s three standard axioms; it does not, of course, bound bugs in the kernel itself.

## 9 Reproduction

From the repository root:

```
lake build RequestProject.ZkPoP      -- builds the whole layer  
lake build RequestProject.ZkPoP.Audit -- prints every headline axiom closure  
rg -n "sorry" RequestProject/ZkPoP/  -- expected: no matches
```

A phase-by-phase index from the design document to the Lean artifacts is in docs/ZKPOP\_SOP.md; the L<sup>A</sup>T<sub>E</sub>X source of this report is in docs/, next to it.